UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias 2026
Introducción a Ciencias de la Computación

REPORTE — PRÁCTICA 7

Introducción a la Programación Orientada a objetos

Nombre: López Pérez Luis Ángel
Número de cuenta: 324243174
Sistema operativo: Linux
Distribución: Ubuntu
Fecha: **8 / 11 / 2026**

1. Introducción

En esta práctica modelé y programé en Java el sistema de una empresa de entrega de paquetes. A partir del enunciado realicé a mano el análisis gramatical, obtuve la clase Paquete y la implementé junto con la clase de utilería CotizadorEnvio, que calcula el costo del envío por rangos de peso y determina si el paquete requiere seguro obligatorio. Finalmente lo probé todo con la clase PruebaPaqueteria.

Organicé este reporte siguiendo el orden de la práctica: Parte 1 (análisis y modelado), Parte 2 (clase Paquete), Parte 3 (clase CotizadorEnvio) y Parte 4 (pruebas), y lo cierro con los comandos de compilación, ejecución y generación de la documentación.

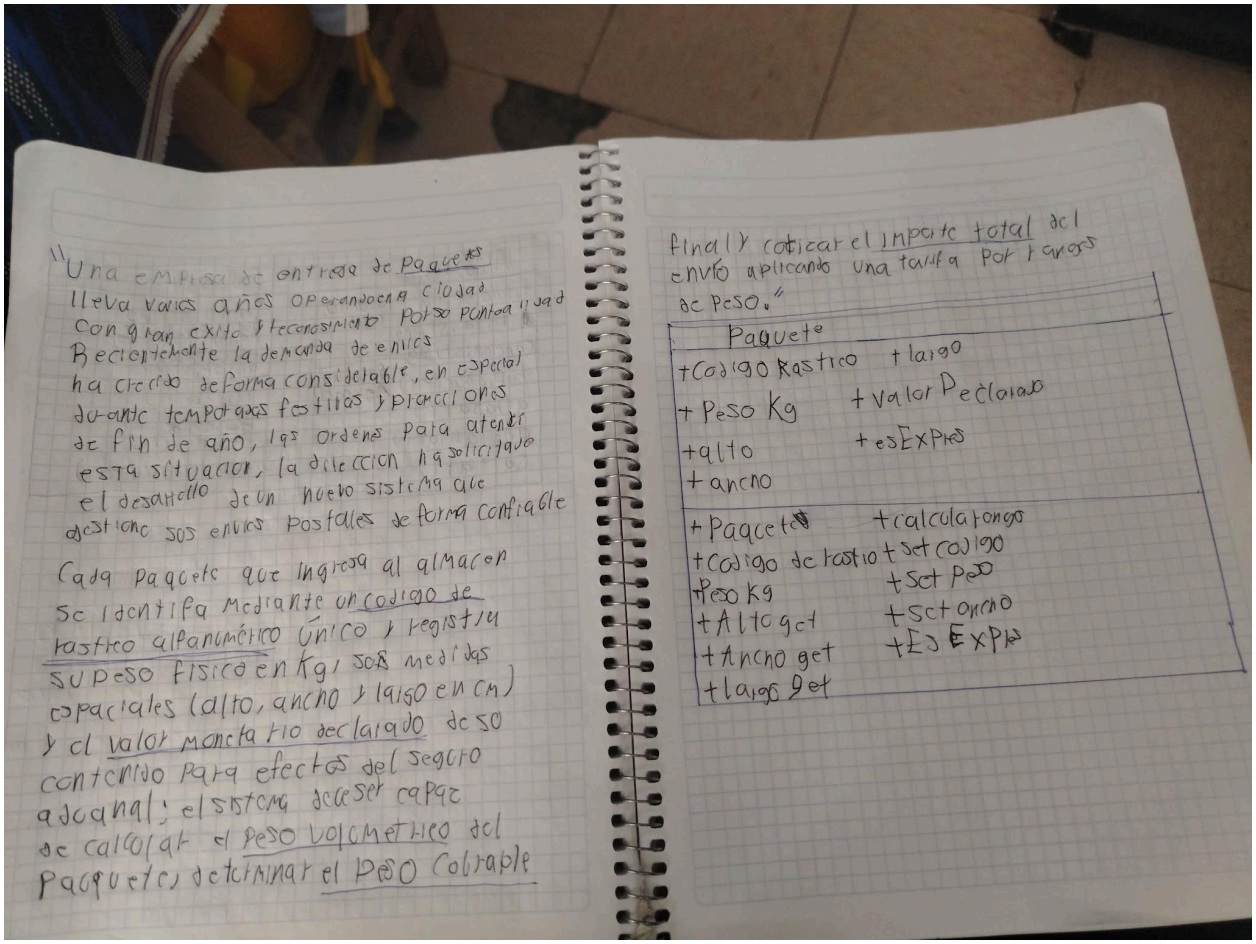
2. Parte 1: Análisis gramatical y modelado

2.1 Análisis gramatical

Realicé el análisis gramatical completo a mano en una hoja de papel, que entrego en físico al inicio de la sesión. La siguiente tabla resume mis resultados: qué sustantivos y verbos del texto convertí en clase, atributo o método, y cuáles descarté.

Elemento del texto	Categoría gramatical	Decisión de modelado
paquete	Sustantivo (sujeto principal)	Clase Paquete
código de rastreo alfanumérico	Sustantivo + adjetivos	Atributo codigoRastreo
peso físico en kilogramos	Sustantivo + complemento	Atributo pesoKg
medidas: alto, ancho y largo en centímetros	Sustantivos	Atributos alto, ancho, largo
valor monetario	Sustantivo + adjetivo	Atributo valorDeclarado
calcular el peso volumétrico	Verbo + complemento directo	Método calculaPesoVolumetrico()
determinar el peso cobrable final	Verbo + complemento directo	Método obtenerPesoCobrable()
cotizar el importe total	Verbo + complemento directo	Lo delego a CotizadorEnvio.cotizarEnvio(); es una regla de negocio, no un dato del paquete
empresa, ciudad, dirección, personal administrativo, almacén, sistema, órdenes, temporadas	Relleno	Descartados (ver 2.3)

Figura 1. Hoja con el análisis gramatical hecho a mano y el diagrama de clase



2.2 Diagrama de clase

Del análisis obtuve una sola clase de datos, Paquete, que represento en la siguiente tabla (el signo − indica visibilidad privada y + visibilidad pública):

Paquete
Atributos - codigoRastreo : String - pesoKg : double - alto : double - ancho : double - largo : double - valorDeclarado : double - esExpres : boolean (propuesta personal, Parte 2)

Métodos
+ Paquete()
+ Paquete(codigoRastreo : String, pesoKg : double, alto : double, ancho : double, largo : double, valorDeclarado : double)
+ getCodigoRastreo() : String + setCodigoRastreo(String) : void
+ getPesoKg() : double + setPesoKg(double) : void
+ getAlto() : double + setAlto(double) : void
+ getAncho() : double + setAncho(double) : void
+ getLargo() : double + setLargo(double) : void
+ getValorDeclarado() : double + setValorDeclarado(double) : void
+ isEsExpres() : boolean + setEsExpres(boolean) : void
+ calculaPesoVolumetrico() : double
+ obtenerPesoCobrable() : double
+ calculaRecargoExpres(importeBase : double) : double

Además agregué la clase de utilería CotizadorEnvio, sin atributos, con tres métodos estáticos: esPaqueteValido(Paquete), cotizarEnvio(Paquete) y requiereSeguro(Paquete) (Parte 3).

2.3 Elementos descartados y justificación

Del enunciado descarté los siguientes elementos porque no aportan datos ni comportamiento al paquete:

- **Empresa, ciudad, dirección y personal administrativo:** describen el contexto del problema o a los usuarios del sistema, no al paquete. La práctica no pide modelarlos como clases y no los necesito para calcular peso, tarifa ni seguro.
- **Éxito, reconocimiento, puntualidad, demanda, temporadas festivas y promociones:** son motivaciones narrativas del problema, no tienen un valor concreto que mi programa deba almacenar o calcular.
- **Almacén, órdenes y sistema:** son entidades más amplias (inventario, seguimiento de pedidos) que quedan fuera del alcance, así que limité mi modelo a la cotización de un paquete individual.
- **Seguro aduanal:** no lo modelé como clase; solo me interesa su efecto, que es el valor declarado del contenido (atributo) y la regla que decide si el seguro es obligatorio (método de CotizadorEnvio).
- **Tarifa por rangos de peso:** no es un atributo de un paquete en particular, sino una regla compartida por todos, por lo que la coloqué en CotizadorEnvio y no en Paquete.
- **Unicidad del código de rastreo:** garantizarla requiere conocer todos los paquetes del sistema, algo que una clase individual no puede hacer. Por eso en Paquete solo valido que el código no esté vacío.

2.4 Criterios para elegir el tipo de dato

Atributo	Tipo	Criterio
codigoRastreo	String	Es alfanumérico, así que no puede ser un número. Además no se hacen operaciones aritméticas con él, solo se identifica y se compara.
pesoKg	double	El peso puede tener fracciones (por ejemplo 2.5 kg). Un int truncaría el valor y alteraría la cotización.
alto, ancho, largo	double	Las medidas en centímetros pueden ser decimales y participan en una división (÷ 5000), por lo que se necesita precisión fraccionaria.
valorDeclarado	double	Es un monto en pesos con centavos que se compara con $3,000.00. Uso double por simplicidad y por lo visto en el curso; en un sistema real convendría BigDecimal para evitar errores de redondeo.
esExpres	boolean	Solo tiene dos estados posibles el envío es exprés o no.

3. Parte 2: Implementación de la clase Paquete

3.1 Atributos y constructores

Los atributos son los de la tabla de la sección 2.2; los declaré todos private para respetar el encapsulamiento: solo se leen y modifican mediante getters y setters, donde puedo validar el dato.

Constructor por defecto. Inicializa el paquete con valores predeterminados válidos: código "SIN-CODIGO", peso 1.0 kg, alto, ancho y largo de 1.0 cm, valor declarado 0.0 y envío no exprés. Así evito que exista un paquete con dimensiones nulas o negativas.

Constructor con parámetros. Recibe los datos iniciales y valida cada uno. Las validaciones que incluí y su justificación son:

Dato	Validación	Por qué
codigoRastreo	No nulo y no vacío (después de quitar espacios)	Un paquete sin identificador no se podría rastrear; además evita un NullPointerException al consultarlo.
pesoKg	Mayor que 0	Un paquete físico no puede pesar 0 ni un valor negativo.
alto, ancho, largo	Cada uno mayor que 0	Una dimensión de 0 o negativa no tiene sentido físico y haría que el peso volumétrico sea 0 o negativo, falseando el peso cobrable.

Dato	Validación	Por qué
valorDeclarado	Mayor o igual que 0	Puede declararse 0, pero un valor monetario negativo no es válido para el seguro.

Cuando un dato no cumple la validación, el atributo conserva su valor por defecto en lugar de abortar el programa con una excepción; así el objeto siempre queda en un estado consistente. Apliqué la misma validación en los setters: si el valor nuevo es inválido, el atributo conserva su valor anterior.

3.2 Métodos de consulta y modificación

Agregué un getter y un setter por cada atributo (por ejemplo getPesoKg() y setPesoKg(double)). Los getters solo devuelven el valor y los setters validan antes de asignar, de modo que ningún código externo pueda dejar al paquete en un estado inválido.

3.3 Métodos calculadores base

calculaPesoVolumetrico(). Convierte el volumen en cm³ a un peso equivalente con la fórmula estándar del transporte comercial (norma IATA):

pesoVolumetrico = (largo * ancho * alto) / 5000

El divisor 5000 es el factor de conversión de cm³ a kg. Existe porque en las bodegas el espacio es limitado: un paquete ligero pero voluminoso (de ropa o unicel) ocupa lugar como si pesara más, y debe cobrarse por ello.

obtenerPesoCobrable(). Devuelve el mayor entre el peso real y el volumétrico, con Math.max(pesoKg, calculaPesoVolumetrico()).

Ejemplo de verificación. Un paquete de 4 kg que mide 40 × 30 × 50 cm tiene volumen 60,000 cm³; su peso volumétrico es 60,000 / 5000 = 12 kg. Como 12 > 4, el peso cobrable es 12 kg.

3.4 Propuesta de diseño personal: envío exprés

Requerimiento imaginado. Imaginé que la empresa quiere ofrecer entrega urgente (exprés) con un recargo del 30 % sobre el importe base del envío. Me parece un requerimiento realista: en temporadas festivas, que el propio enunciado menciona, muchos clientes piden urgencia.

- **Atributo nuevo:** private boolean esExpres (con getter isEsExpres() y setter setEsExpres()). Por defecto es false.
- **Método nuevo:** public double calculaRecargoExpres(double importeBase). Si el paquete es exprés y el importe base es mayor que 0, devuelve importeBase × 0.30; en cualquier otro caso devuelve 0.0.

Decidí que el método reciba el importe base como parámetro, en lugar de calcularlo internamente, para mantener la separación de responsabilidades: la tarifa por rangos vive en CotizadorEnvio y Paquete solo sabe si su envío es exprés. De esta forma Paquete no depende de las reglas de tarifa, que podrían cambiar.

Ejemplo: el paquete anterior (12 kg cobrables) cuesta $246.00 de envío base (ver sección 4); con exprés el recargo es 246.00 × 0.30 = $73.80 y el total $319.80.

4. Parte 3: Implementación de la clase CotizadorEnvio

CotizadorEnvio es una clase de utilería: no guarda datos propios, solo contiene reglas de negocio compartidas. Por eso sus tres métodos son public static y reciben el paquete como parámetro.

4.1 esPaqueteValido(Paquete p)

Devuelve true solo si la referencia no es null (y, como protección adicional, si el peso y las tres dimensiones del paquete son mayores que 0). La comprobación de null va primero porque consultar las propiedades de una referencia nula provoca un NullPointerException.

Un paquete creado con el constructor por defecto sí es un objeto válido: existe en memoria y sus valores predeterminados son consistentes. Por eso esPaqueteValido lo acepta, a diferencia de un null.

4.2 cotizarEnvio(Paquete p)

Si p no es válido devuelve -1.0 como código de error. En caso contrario toma el peso cobrable y aplica la tarifa escalonada:

Peso cobrable	Importe
Hasta 5 kg	$120.00 (cuota fija)
Más de 5 kg y hasta 15 kg	$120.00 + $18.00 por cada kg que exceda 5 kg
Más de 15 kg	$300.00 + $25.00 por cada kg que exceda 15 kg

Comprobé la tarifa a mano con estos ejemplos:

- 4 kg cobrables → $120.00 (cuota fija).
- 12 kg cobrables → 120 + 18 × (12 − 5) = 120 + 126 = $246.00.
- 15 kg cobrables → 120 + 18 × 10 = $300.00, que coincide con el inicio del siguiente tramo, así que la tarifa es continua.
- 20 kg cobrables → 300 + 25 × (20 − 15) = 300 + 125 = $425.00.

4.3 requiereSeguro(Paquete p)

Devuelve false si el paquete no es válido. Si lo es, devuelve true cuando se cumple al menos una de las siguientes condiciones (unidas con el operador ||):

- El valor declarado es mayor que $3,000.00.
- Es sobredimensionado: alguna arista (alto, ancho o largo) excede 60 cm, o la suma de las tres dimensiones excede 150 cm.

Los límites son estrictos (excede, supera): un valor de exactamente $3,000.00 o una arista de exactamente 60 cm no activan el seguro.

4.4 Preguntas del reporte

¿Por qué resulta conveniente que CotizadorEnvio tenga métodos estáticos en lugar de requerir crear instancias con new?

En mi diseño, CotizadorEnvio no tiene estado: no guarda atributos que cambien de un objeto a otro, y cada método obtiene todo lo que necesita de sus parámetros. Crear una instancia no aportaría nada, ya que todas serían idénticas, y gastaría memoria y líneas de código innecesarias. Con métodos estáticos la regla de negocio queda en un solo lugar y se invoca directamente con el nombre de la clase (CotizadorEnvio.cotizarEnvio(p)), igual que Math.max(), que tampoco se instancia.

¿Qué diferencia conceptual y práctica existe entre una variable de referencia con valor null y un objeto instanciado con su constructor por defecto?

Una referencia null no apunta a ningún objeto: es una variable que existe, pero no hay nada en memoria detrás de ella. Un objeto creado con el constructor por defecto sí existe: tiene espacio en memoria y sus atributos contienen valores definidos, aunque sean genéricos.

5. Parte 4: Pruebas y verificación

5.1 Casos de prueba proporcionados

En PruebaPaqueteria.java instancié p1, p2, p3 y p4 con new y el constructor con parámetros, con los datos indicados en los comentarios de cada caso. La siguiente tabla resume los resultados que obtuve:

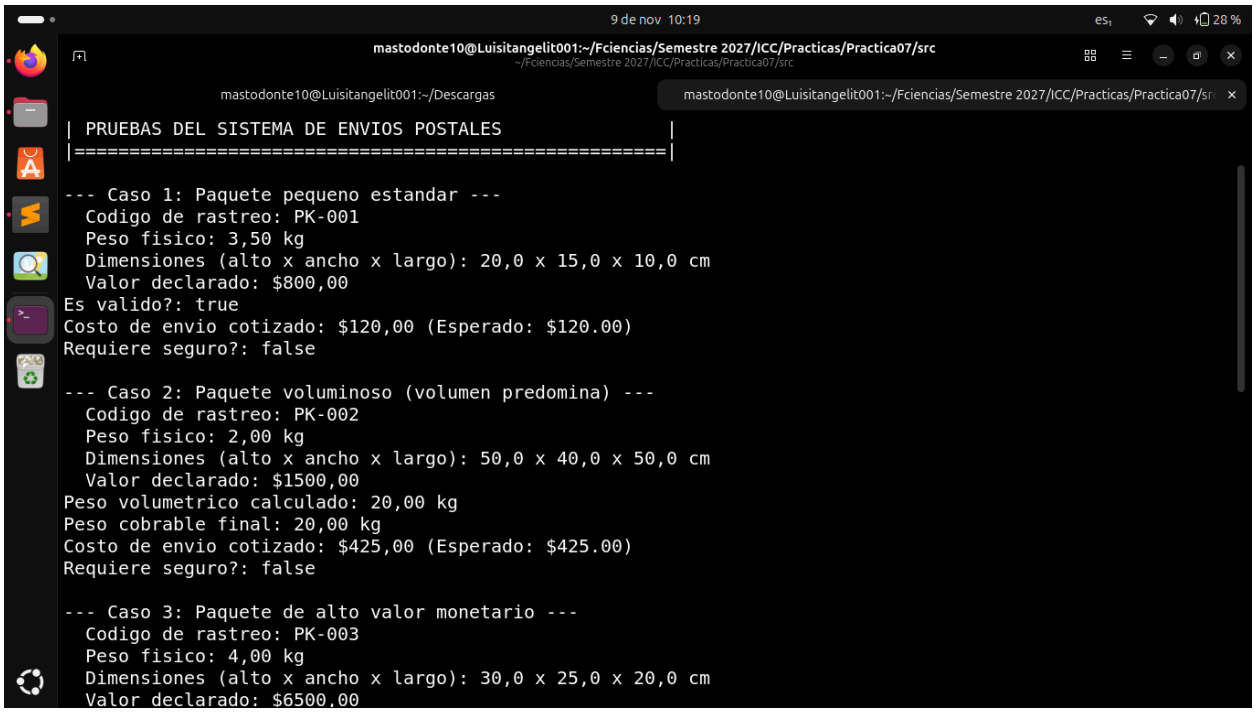
Caso	Datos	Peso vol.	Peso cobrable	Cotización	¿Seguro?
p1	PK-001	0.60 kg	3.50 kg	$120.00	No
p2	PK-002	20.00 kg	20.00 kg	$425.00	No
p3	PK-003	3.00 kg	4.00 kg	$120.00	Sí
p4	PK-004	8.40 kg	8.40 kg	$181.20	Sí

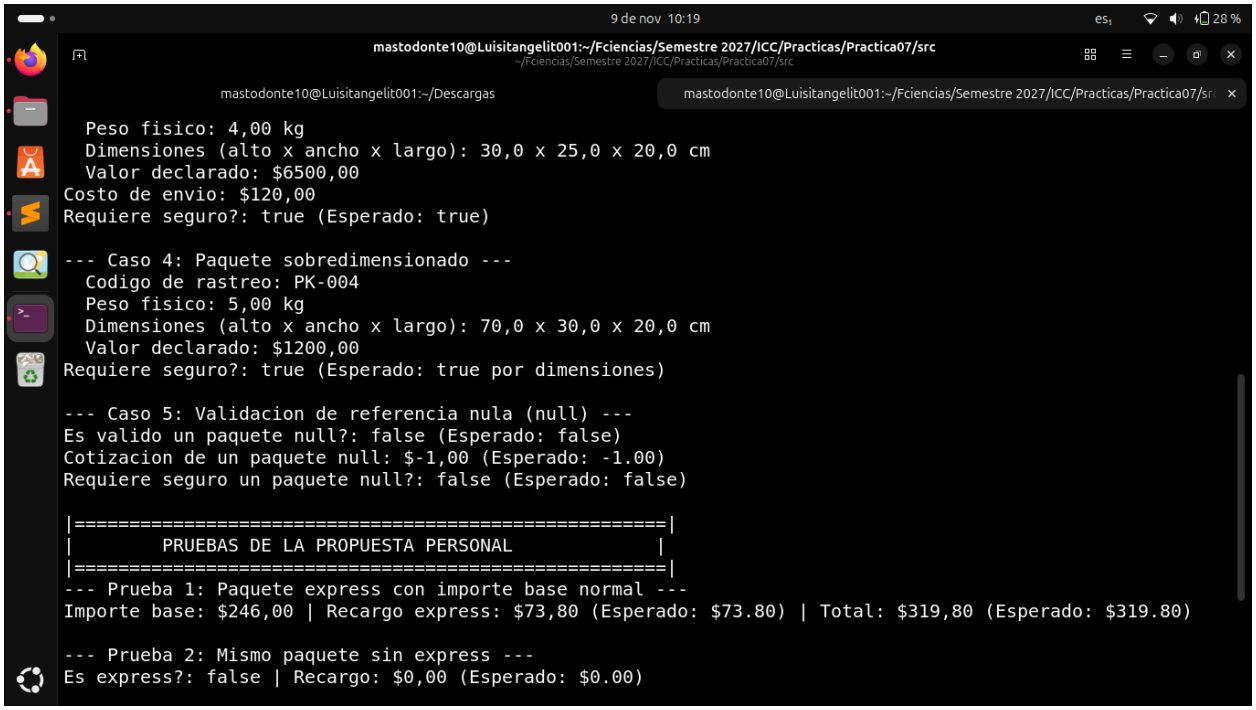
Qué válida cada caso:

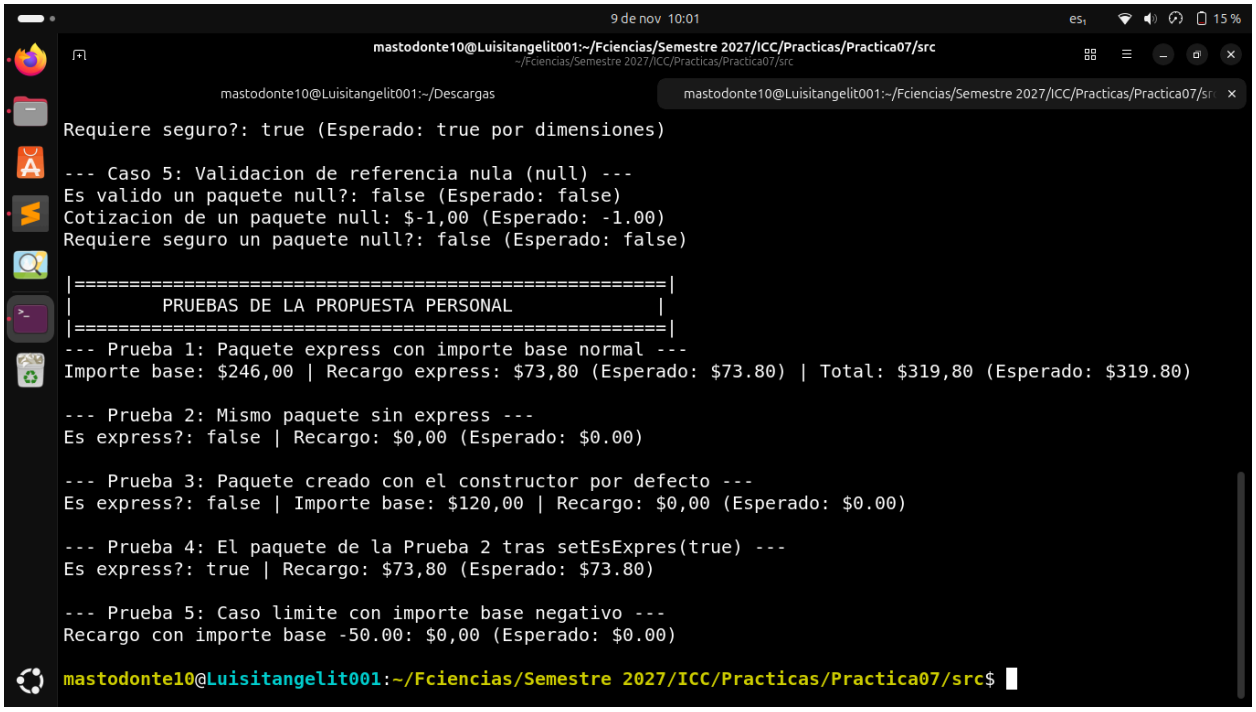
- **p1:** el peso físico (3.5 kg) domina sobre el volumétrico (0.6 kg), así que se cobra la cuota fija de $120.00 y no hay seguro.
- **p2:** el paquete es ligero (2 kg) pero voluminoso: su peso volumétrico (20 kg) domina y se cobra $300 + 25 \times (20 - 15) = \$425.00$. No requiere seguro porque ninguna arista pasa de 60 cm y la suma de dimensiones (140 cm) no excede 150 cm.
- **p3:** pesa más que su volumen (4 kg contra 3 kg), así que se cobra $120.00; requiere seguro porque su valor declarado ($6,500.00) supera $3,000.00.

- **p4:** el peso volumétrico (8.4 kg) domina sobre el real (5 kg), por lo que se cobra 120 + 18 × (8.4 − 5) = $181.20; requiere seguro por sobredimensionado, ya que una arista (70 cm) excede 60 cm.
- **Caso 5 (null):** esPaqueteValido devuelve false, cotizarEnvio devuelve −1.00 y requiereSeguro devuelve false, sin provocar ningún error, porque la validación de null se hace antes de consultar cualquier propiedad.

Todos los resultados coincidieron con los valores esperados que indica la plantilla de PruebaPaqueteria.





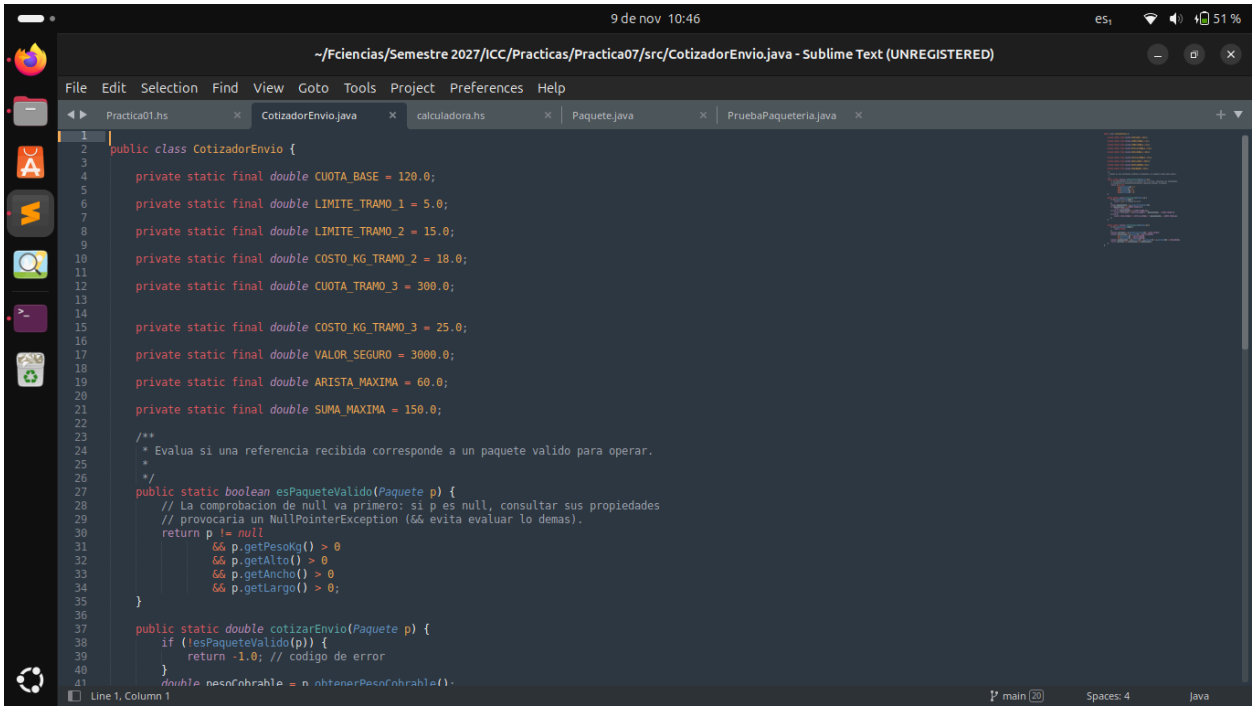


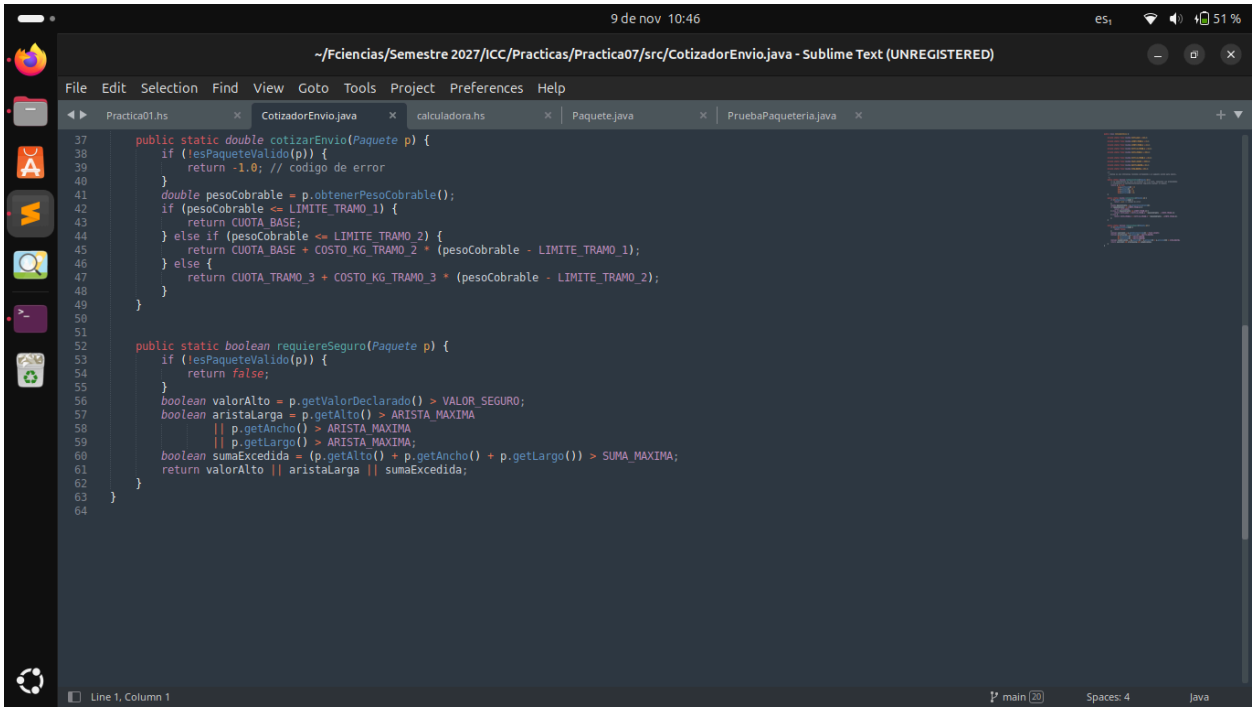
5.2 Pruebas de la propuesta personal (envío exprés)

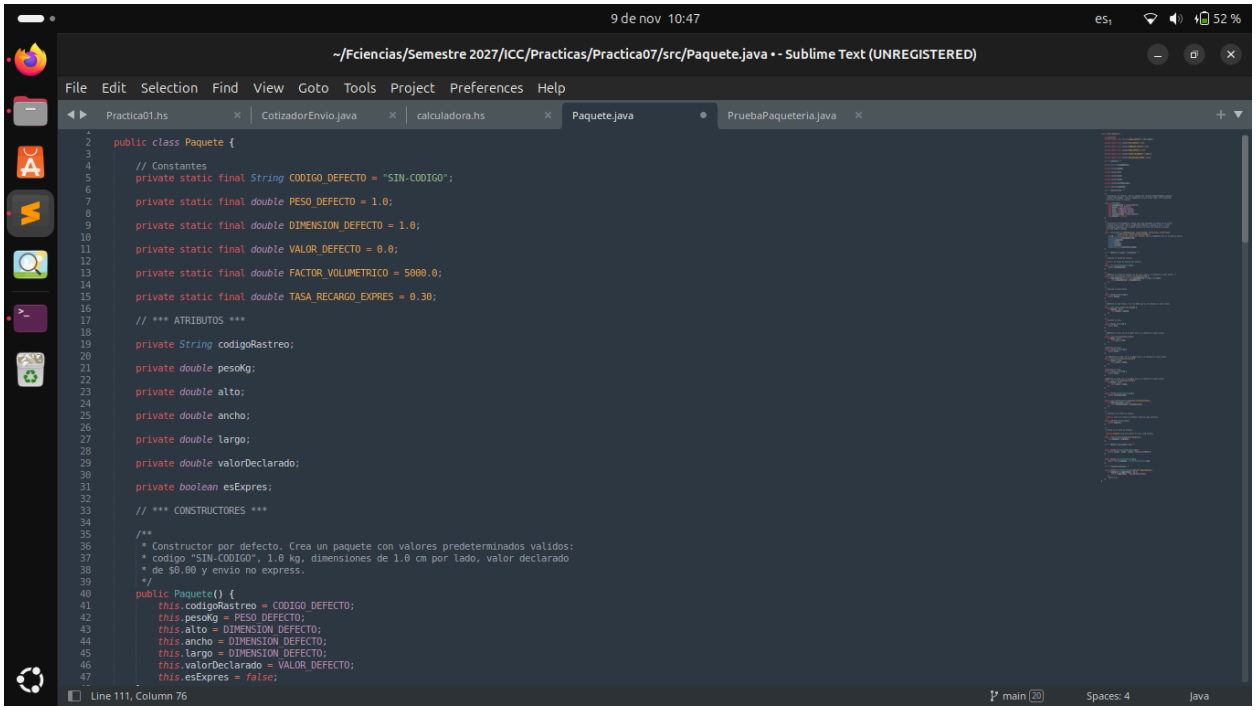
Al final del main de PruebaPaqueteria agregué una sección que instancia paquetes y prueba calculaRecargoExpres(). Elegí los datos para cubrir el caso normal, los casos en que no debe haber recargo y un caso límite:

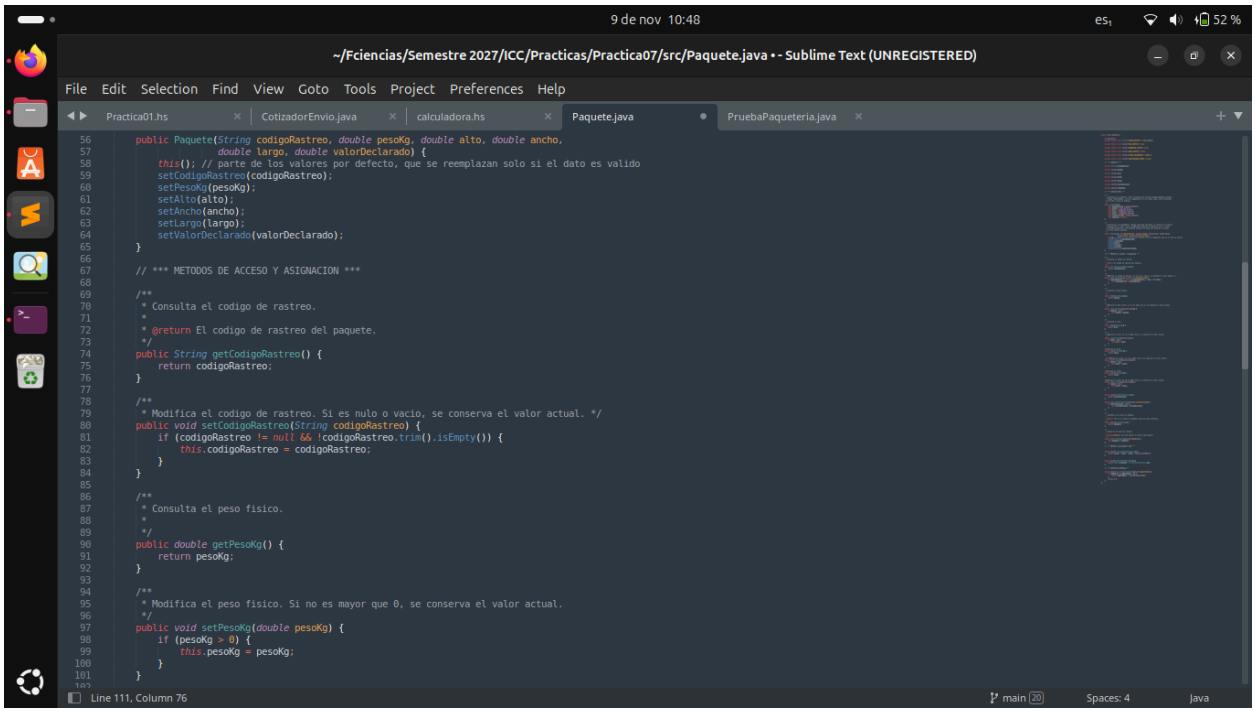
#	Datos de entrada	Resultado esperado	Qué se verifica
1	Paquete de 4 kg, 40×30×50 cm, exprés = true; importe base 246.00	Recargo 73.80 (total 319.80)	El caso principal: el 30 % se calcula bien sobre el importe de la cotización.
2	El mismo paquete, exprés = false; importe base 246.00	Recargo 0.0	Sin exprés no debe haber recargo.
3	Paquete creado con el constructor por defecto	Recargo 0.0	El valor por defecto de esExpres es false.
4	El paquete del caso 2, tras setEsExpres(true)	Recargo 73.80	El setter cambia el comportamiento del objeto.
5	Exprés = true; importe base −50.0	Recargo 0.0	Caso límite: un importe no válido no debe generar un recargo negativo.

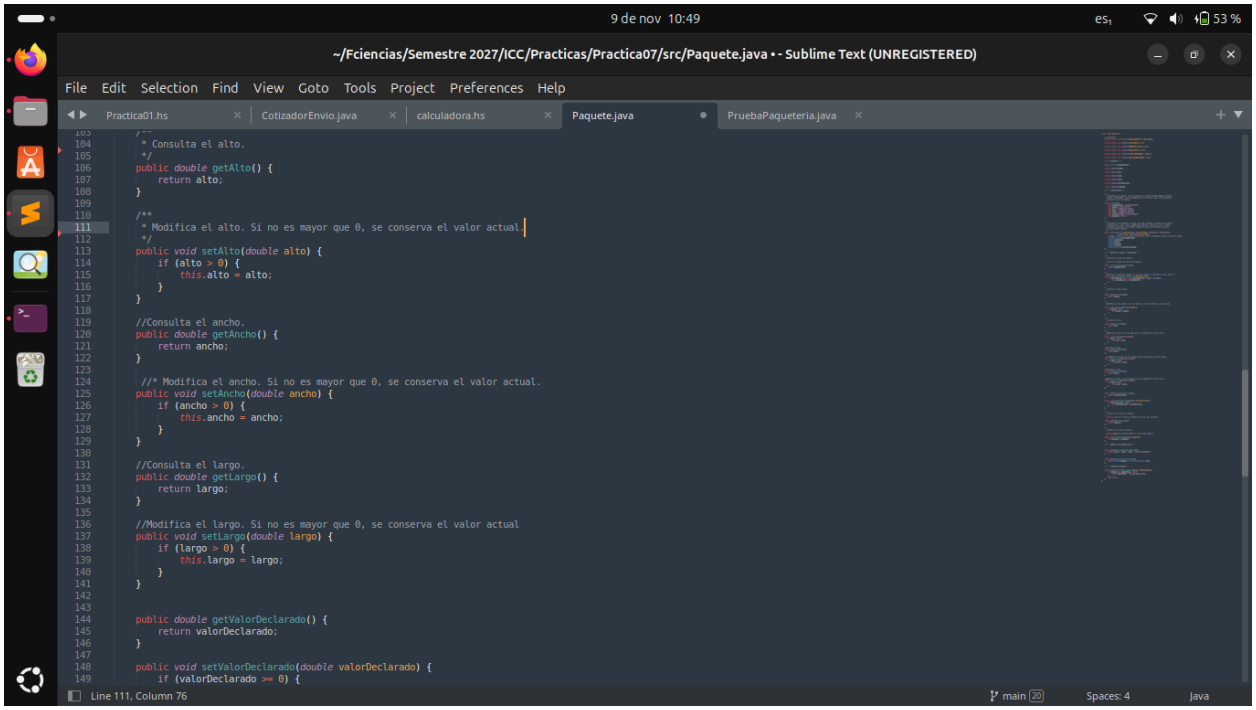
En las cinco pruebas obtuve el resultado esperado.

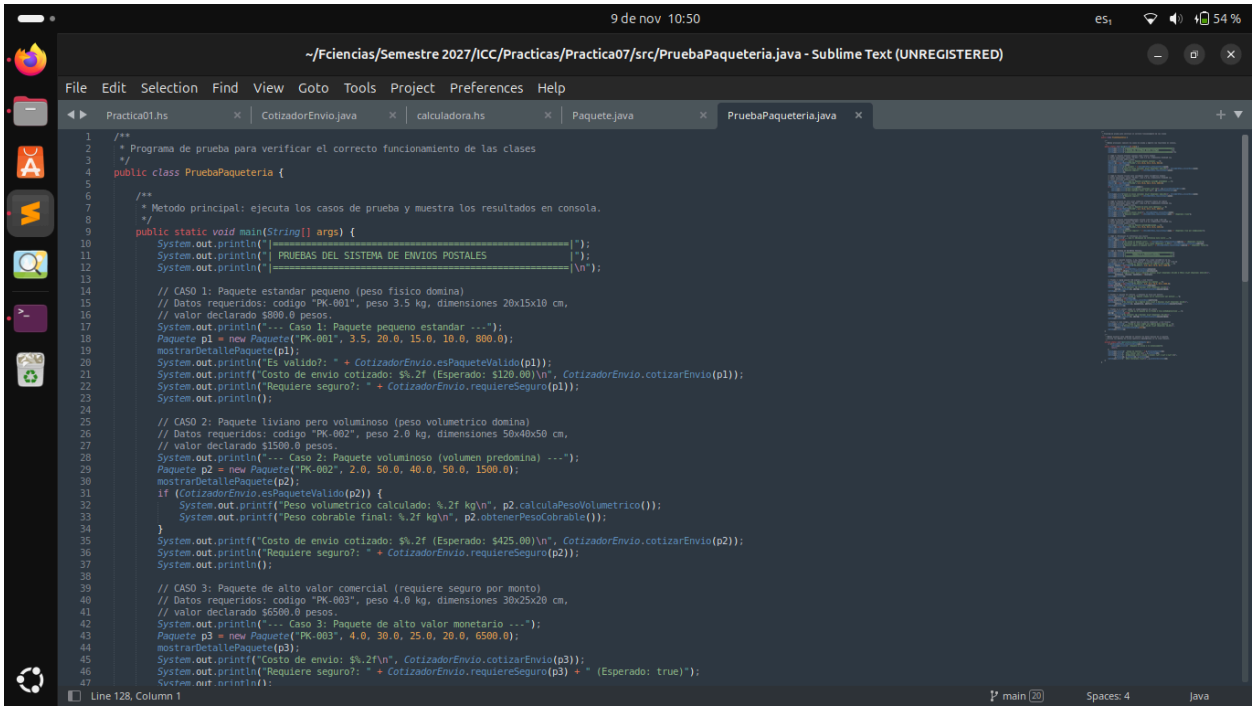

9 de nov 10:46
es
51 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/CotizadorEnvio.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
public class CotizadorEnvio {
private static final double CUOTA_BASE = 120.0;
private static final double LIMITE_TRAMO_1 = 5.0;
private static final double LIMITE_TRAMO_2 = 15.0;
private static final double COSTO_KG_TRAMO_2 = 18.0;
private static final double CUOTA_TRAMO_3 = 300.0;
private static final double COSTO_KG_TRAMO_3 = 25.0;
private static final double VALOR_SEGURO = 3000.0;
private static final double ARISTA_MAXIMA = 60.0;
private static final double SUMA_MAXIMA = 150.0;
/**
* Evalua si una referencia recibida corresponde a un paquete valido para operar.
*
*/
public static boolean esPaqueteValido(Paquete p) {
// La comprobacion de null va primero: si p es null, consultar sus propiedades
// provocaria un NullPointerException (&& evita evaluar lo demas).
return p != null
&& p.getPesoKg() > 0
&& p.getAlto() > 0
&& p.getAncho() > 0
&& p.getLargo() > 0;
}
public static double cotizarEnvio(Paquete p) {
if (!esPaqueteValido(p)) {
return -1.0; // codigo de error
}
Line 1, Column 1
main 20
Spaces: 4
Java

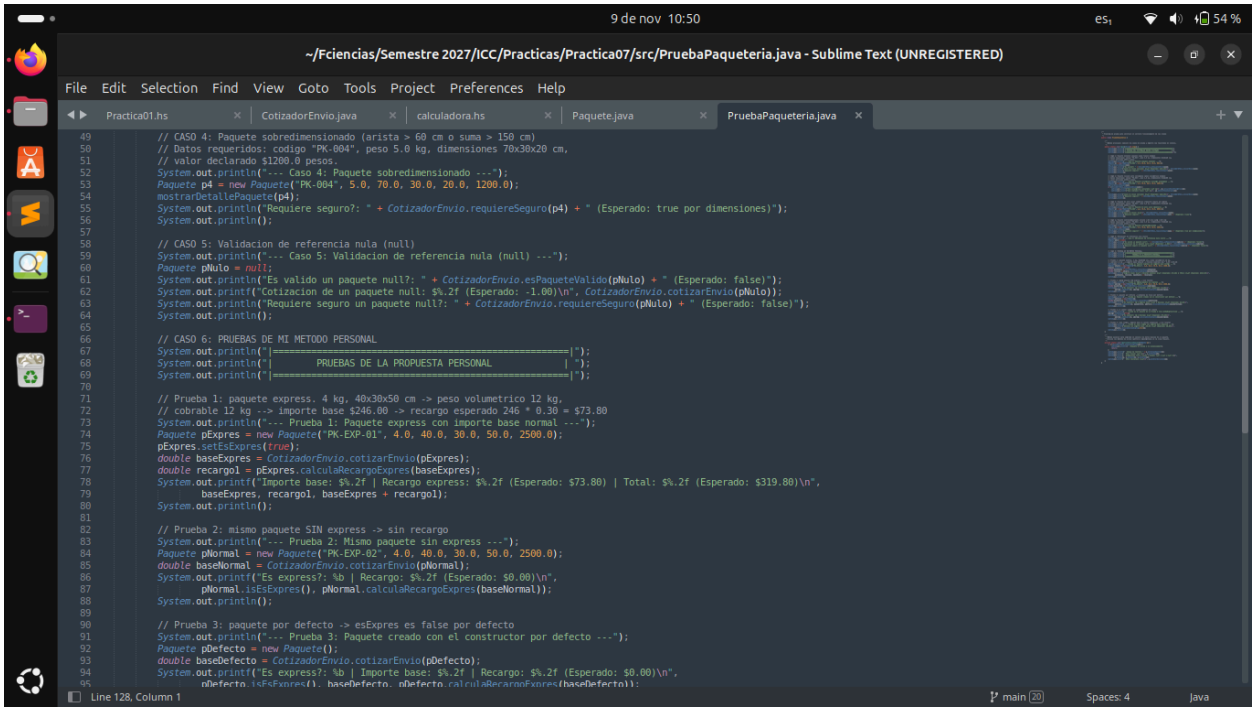

9 de nov 10:46
es
51 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/CotizadorEnvio.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
public static double cotizarEnvio(Paquete p) {
if (!esPaqueteValido(p)) {
return -1.0; // codigo de error
}
double pesoCobrable = p.obtenerPesoCobrable();
if (pesoCobrable <= LIMITE_TRAMO_1) {
return CUOTA_BASE;
} else if (pesoCobrable <= LIMITE_TRAMO_2) {
return CUOTA_BASE + COSTO_KG_TRAMO_2 * (pesoCobrable - LIMITE_TRAMO_1);
} else {
return CUOTA_TRAMO_3 + COSTO_KG_TRAMO_3 * (pesoCobrable - LIMITE_TRAMO_2);
}
}
public static boolean requiereSeguro(Paquete p) {
if (!esPaqueteValido(p)) {
return false;
}
boolean valorAlto = p.getValorDeclarado() > VALOR_SEGURO;
boolean aristaLarga = p.getAlto() > ARISTA_MAXIMA
|| p.getAncho() > ARISTA_MAXIMA
|| p.getLargo() > ARISTA_MAXIMA;
boolean sumaExcedida = (p.getAlto() + p.getAncho() + p.getLargo()) > SUMA_MAXIMA;
return valorAlto || aristaLarga || sumaExcedida;
}
}
Line 1, Column 1
main 20
Spaces: 4
Java

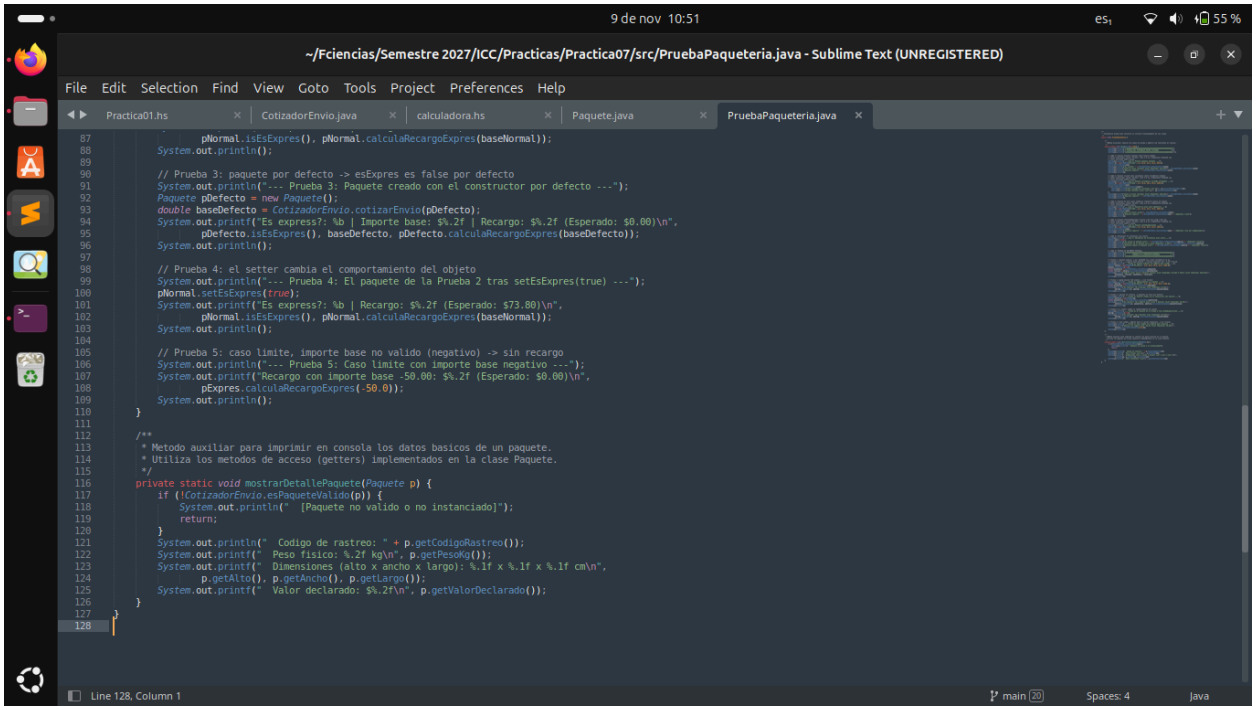

9 de nov 10:47
es
52 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/Paquete.java • - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
public class Paquete {
// Constantes
private static final String CODIGO_DEFECTO = "SIN-CODIGO";
private static final double PESO_DEFECTO = 1.0;
private static final double DIMENSION_DEFECTO = 1.0;
private static final double VALOR_DEFECTO = 0.0;
private static final double FACTOR_VOLUMETRICO = 5000.0;
private static final double TASA_RECARGO_EXPRES = 0.30;
// *** ATRIBUTOS ***
private String codigoRastreo;
private double pesoKg;
private double alto;
private double ancho;
private double largo;
private double valorDeclarado;
private boolean esExpres;
// *** CONSTRUCTORES ***
/**
* Constructor por defecto. Crea un paquete con valores predeterminados validos:
* codigo "SIN-CODIGO", 1.0 kg, dimensiones de 1.0 cm por lado, valor declarado
* de $0.00 y envio no express.
*/
public Paquete() {
this.codigoRastreo = CODIGO_DEFECTO;
this.pesoKg = PESO_DEFECTO;
this.alto = DIMENSION_DEFECTO;
this.ancho = DIMENSION_DEFECTO;
this.largo = DIMENSION_DEFECTO;
this.valorDeclarado = VALOR_DEFECTO;
this.esExpres = false;
Line 111, Column 76
main 20
Spaces: 4
Java


9 de nov 10:48
es
52 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/Paquete.java • - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
public Paquete(String codigoRastreo, double pesoKg, double alto, double ancho,
double largo, double valorDeclarado) {
this(); // parte de los valores por defecto, que se reemplazan solo si el dato es valido
setCodigoRastreo(codigoRastreo);
setPesoKg(pesoKg);
setAlto(alto);
setAncho(ancho);
setLargo(largo);
setValorDeclarado(valorDeclarado);
}
// *** METODOS DE ACCESO Y ASIGNACION ***
/**
* Consulta el codigo de rastreo.
*
* @return El codigo de rastreo del paquete.
*/
public String getCodigoRastreo() {
return codigoRastreo;
}
/**
* Modifica el codigo de rastreo. Si es nulo o vacio, se conserva el valor actual. */
public void setCodigoRastreo(String codigoRastreo) {
if (codigoRastreo != null && !codigoRastreo.trim().isEmpty()) {
this.codigoRastreo = codigoRastreo;
}
}
/**
* Consulta el peso fisico.
*
*/
public double getPesoKg() {
return pesoKg;
}
/**
* Modifica el peso fisico. Si no es mayor que 0, se conserva el valor actual.
*/
public void setPesoKg(double pesoKg) {
if (pesoKg > 0) {
this.pesoKg = pesoKg;
}
}
Line 111, Column 76
main 20
Spaces: 4
Java


9 de nov 10:49
es
53 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/Paquete.java • - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
* Consulta el alto.
*/
public double getAlto() {
return alto;
}
/**
* Modifica el alto. Si no es mayor que 0, se conserva el valor actual.
*/
public void setAlto(double alto) {
if (alto > 0) {
this.alto = alto;
}
}
//Consulta el ancho.
public double getAncho() {
return ancho;
}
//* Modifica el ancho. Si no es mayor que 0, se conserva el valor actual.
public void setAncho(double ancho) {
if (ancho > 0) {
this.ancho = ancho;
}
}
//Consulta el largo.
public double getLargo() {
return largo;
}
//Modifica el largo. Si no es mayor que 0, se conserva el valor actual
public void setLargo(double largo) {
if (largo > 0) {
this.largo = largo;
}
}
public double getValorDeclarado() {
return valorDeclarado;
}
public void setValorDeclarado(double valorDeclarado) {
if (valorDeclarado >= 0) {
Line 111, Column 76
main 20
Spaces: 4
Java


9 de nov 10:50
es
54 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/PruebaPaqueteria.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
/**
* Programa de prueba para verificar el correcto funcionamiento de las clases
*/
public class PruebaPaqueteria {
/**
* Metodo principal: ejecuta los casos de prueba y muestra los resultados en consola.
*/
public static void main(String[] args) {
System.out.println("|==|");
System.out.println("| PRUEBAS DEL SISTEMA DE ENVIOS POSTALES |");
System.out.println("|==|\n");
// CASO 1: Paquete estandar pequeno (peso fisico domina)
// Datos requeridos: codigo "PK-001", peso 3.5 kg, dimensiones 20x15x10 cm,
// valor declarado $800.0 pesos.
System.out.println("--- Caso 1: Paquete pequeno estandar ---");
Paquete p1 = new Paquete("PK-001", 3.5, 20.0, 15.0, 10.0, 800.0);
mostrarDetallePaquete(p1);
System.out.println("Es valido?: " + CotizadorEnvio.esPaqueteValido(p1));
System.out.printf("Costo de envio cotizado: $%.2f (Esperado: $120.00)\n", CotizadorEnvio.cotizarEnvio(p1));
System.out.println("Requiere seguro?: " + CotizadorEnvio.requiereSeguro(p1));
System.out.println();
// CASO 2: Paquete liviano pero voluminoso (peso volumetrico domina)
// Datos requeridos: codigo "PK-002", peso 2.0 kg, dimensiones 50x40x50 cm,
// valor declarado $1500.0 pesos.
System.out.println("--- Caso 2: Paquete voluminoso (volumen predomina) ---");
Paquete p2 = new Paquete("PK-002", 2.0, 50.0, 40.0, 50.0, 1500.0);
mostrarDetallePaquete(p2);
if (CotizadorEnvio.esPaqueteValido(p2)) {
System.out.printf("Peso volumetrico calculado: %.2f kg\n", p2.calculaPesoVolumetrico());
System.out.printf("Peso cobrable final: %.2f kg\n", p2.obtenerPesoCobrable());
}
System.out.printf("Costo de envio cotizado: $%.2f (Esperado: $425.00)\n", CotizadorEnvio.cotizarEnvio(p2));
System.out.println("Requiere seguro?: " + CotizadorEnvio.requiereSeguro(p2));
System.out.println();
// CASO 3: Paquete de alto valor comercial (requiere seguro por monto)
// Datos requeridos: codigo "PK-003", peso 4.0 kg, dimensiones 30x25x20 cm,
// valor declarado $6500.0 pesos.
System.out.println("--- Caso 3: Paquete de alto valor monetario ---");
Paquete p3 = new Paquete("PK-003", 4.0, 30.0, 25.0, 20.0, 6500.0);
mostrarDetallePaquete(p3);
System.out.printf("Costo de envio: $%.2f\n", CotizadorEnvio.cotizarEnvio(p3));
System.out.println("Requiere seguro?: " + CotizadorEnvio.requiereSeguro(p3) + " (Esperado: true)");
System.out.println();
Line 128, Column 1
main 20
Spaces: 4
Java


9 de nov 10:50
es
54 %
~/Fciencias/Semestre 2027/ICC/Practicas/Practica07/src/PruebaPaqueteria.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
Practica01.hs
CotizadorEnvio.java
calculadora.hs
Paquete.java
PruebaPaqueteria.java
// CASO 4: Paquete sobredimensionado (arista > 60 cm o suma > 150 cm)
// Datos requeridos: codigo "PK-004", peso 5.0 kg, dimensiones 70x30x20 cm,
// valor declarado $1200.0 pesos.
System.out.println("--- Caso 4: Paquete sobredimensionado ---");
Paquete p4 = new Paquete("PK-004", 5.0, 70.0, 30.0, 20.0, 1200.0);
mostrarDetallePaquete(p4);
System.out.println("Requiere seguro?: " + CotizadorEnvio.requiereSeguro(p4) + " (Esperado: true por dimensiones)");
System.out.println();
// CASO 5: Validacion de referencia nula (null)
System.out.println("--- Caso 5: Validacion de referencia nula (null) ---");
Paquete pNulo = null;
System.out.println("Es valido un paquete null?: " + CotizadorEnvio.esPaqueteValido(pNulo) + " (Esperado: false)");
System.out.printf("Cotizacion de un paquete null: $%.2f (Esperado: -1.00)\n", CotizadorEnvio.cotizarEnvio(pNulo));
System.out.println("Requiere seguro un paquete null?: " + CotizadorEnvio.requiereSeguro(pNulo) + " (Esperado: false)");
System.out.println();
// CASO 6: PRUEBAS DE MI METODO PERSONAL
System.out.println("|==|");
System.out.println("| PRUEBAS DE LA PROPUESTA PERSONAL |");
System.out.println("|==|");
// Prueba 1: paquete express. 4 kg, 40x30x50 cm -> peso volumetrico 12 kg,
// cobrable 12 kg --> importe base $246.00 -> recargo esperado 246 * 0.30 = $73.80
System.out.println("--- Prueba 1: Paquete express con importe base normal ---");
Paquete pExpres = new Paquete("PK-EXP-01", 4.0, 40.0, 30.0, 50.0, 2500.0);
pExpres.setEsExpres(true);
double baseExpres = CotizadorEnvio.cotizarEnvio(pExpres);
double recargo1 = pExpres.calculaRecargoExpres(baseExpres);
System.out.printf("Importe base: $%.2f | Recargo express: $%.2f (Esperado: $73.80) | Total: $%.2f (Esperado: $319.80)\n",
baseExpres, recargo1, baseExpres + recargo1);
System.out.println();
// Prueba 2: mismo paquete SIN express -> sin recargo
System.out.println("--- Prueba 2: Mismo paquete sin express ---");
Paquete pNormal = new Paquete("PK-EXP-02", 4.0, 40.0, 30.0, 50.0, 2500.0);
double baseNormal = CotizadorEnvio.cotizarEnvio(pNormal);
System.out.printf("Es express?: %b | Recargo: $%.2f (Esperado: $0.00)\n",
pNormal.isEsExpres(), pNormal.calculaRecargoExpres(baseNormal));
System.out.println();
// Prueba 3: paquete por defecto -> esExpres es false por defecto
System.out.println("--- Prueba 3: Paquete creado con el constructor por defecto ---");
Paquete pDefecto = new Paquete();
double baseDefecto = CotizadorEnvio.cotizarEnvio(pDefecto);
System.out.printf("Es express?: %b | Importe base: $%.2f | Recargo: $%.2f (Esperado: $0.00)\n",
Line 128, Column 1
main 20
Spaces: 4
Java



7. Conclusiones

Lo cierto es que esta ha sido con diferencia esta ha sido la practica mas complicada que hemos hecho hasta ahora y sumado con el paro resulto extremadamente dificil sin los conceptos básicos por lo que se presentaron grandes dificultades para avanzar aunque con ayudas se volvio posible completar la práctica.